

CODECRAFT

**A PROJECT REPORT
for
Major Project-II (CA401P)
Session (2025-26)**

Submitted by

Divyam Raj
(202410116100066)

Divakar Verma
(202410116100065)

Divyanshu Mishra
(202410116100068)

**Submitted in partial fulfilment of the
Requirements for the Degree of**

MASTER OF COMPUTER APPLICATION

**Under the Supervision of
Ms. Annu Yadav
Assistant Professor**



Submitted to

**DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206**

(June 2026)

CERTIFICATE

Certified that **Divyam Raj (202410116100066)**, **Divakar Verma (202410116100065)** and **Divyanshu Mishra (202410116100068)** have carried out the project work having “**CodeCraft**” (CA401P) from KIET Group Of Institutions (an Autonomous college) under Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself, and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Date:

Divyam Raj (202410116100066)

Divakar Verma (202410116100065)

Divyanshu Mishra (202410116100068)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Ms. Annu Yadav
Assistant Professor
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

Dr. Sachin Malhotra
Dean (MCA)
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

ACKNOWLEDGEMENTS

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Ms. Annu Yadav** for her guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions.

Words are not enough to express my gratitude to Dr. Sachin Malhotra, Professor and Dean, Department of Computer Applications, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kind of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Divyam Raj (202410116100066)

Divakar Verma (202410116100065)

Divyanshu Mishra (202410116100068)

TABLE OF CONTENTS

S. No.	Title	Page No.
	Certificate	(ii)
	Acknowledgement	(iii)
	Table of Contents	(iv)
	List of Tables	(ix)
	List of Figures	(x)
	CHAPTER 1: Introduction	1-5
1.1.	Background of the Problem	1
1.2.	Problem Statement	2
1.3	Aims and Objectives	2
1.4	Scope of Project	3
1.5	Hardware and Software requirement	4
	CHAPTER 2: Introduction to Existing System	6-9
2.1.	Overview of Existing Interview Preparation methods	6
2.2	Existing Web Based Platforms	6
2.3	Limitations of Existing Systems	7
2.4	Need for an Intelligent Interview System	8
	CHAPTER 3: Proposed System	10-13

3.1.	Overview of Proposed System	10
3.2.	Core Idea Behind the System	10
3.3.	Detailed Features of the Proposed Systems	11
3.4	System Objectives Fulfilled by the Proposed Solution	12
3.5	Uniqueness of the Proposed System	13
3.6	Advantage of the Proposed System	13
3.7	Summary	13
	CHAPTER 4: System Architecture	14-19
4.1	Introduction to System Architecture	14
4.2	High Level Architecture Overview	14
4.3	Key Architectural Layers	15
4.4	Detailed Module-Level Architecture	17
4.5	Data Flow Description	18
4.6	Architectural Design Principles Used	18
4.7	Technology Stack Justification	18
4.8	Summary	19
	CHAPTER 5: Technology Stack	20-25
5. 1	Introduction to Technology Stack	20
5.2.	Frontend Technologies	20
5.3.	Backend Technologies	21
5.4	Authentication Technology	22
5.5	Artificial Intelligence Technology	23
5.6	Database Technology	23

5.7	Additional Tools and Libraries	24
5.8	Technology Stack Summary	24
	CHAPTER 6: System Design	26-33
6.1.	Introduction to System Design	26
6.2.	Functional Design of the System	26
6.3.	Use Case Descriptions	27
6.4.	Data Flow Description	28
6.5	Entity Relationship Design	30
6.6	Database Design Considerations	31
6.7	UML Design Explanation	31
6.8	Input Design	32
6.9	Output Design	32
6.10	Security Design	33
6.11	Summary	33
	CHAPTER 7: Implementation	34-37
7.1	Introduction to Implementation	34
7.2	Frontend Implementation	34
7.3	Authentication Implementation	35
7.4	Backend Implementation	35
7.5	AI Integration Implementation	36
7.6	Database Implementation	37
7.7	Implementation of Session History	37
7.8	Security Implementation	37
7.9	Challenges Faced During Implementation	37
7.10	Summary	37
	CHAPTER 8: Application Demonstration / Screenshots	38-44

8.1	Introduction	38
8.2	User Authentication Interface	39
8.3	User Dashboard	40
8.4	Interview Category Selection Screen	41
8.5	Coding Practice interface	42
8.6	AI Chatbot and other Sections	43
	CHAPTER 9: Testing	45-48
9.1	Introduction	45
9.2	Objectives of Testing	45
9.3	Testing Strategy	45
9.4	Unit Testing	46
9.5	Integration Testing	46
9.6	System Testing	46
9.7	Functional Testing	46
9.8	Security Testing	47
9.9	Performance Testing	47
9.10	User Acceptance Testing	47
9.11	Test Case Table	47
9.12	Error Handling & Validation Testing	48
	CHAPTER 10: Results and Analysis	49-51
10.1	Introduction	49
10.2	Functional Result	49
10.3	Analysis of AI Feedback Quality	49
10.4	Performance Analysis	50
10.5	User Interaction Analysis	50
10.6	Comparisons with Traditional Preparation Methods	50
10.7	Achievement of Project Objectives	51
10.8	Limitations Observed	51
	CHAPTER 11: Conclusion	52

	CHAPTER 12: Future Scope	53-54
12.1	Future Enhancements	53
12.2	Scalability and Commercial Potential	53
12.3	Summary	54
	CHAPTER 13: References	55

LIST OF TABLES

Table No.	Name of Table	Page No.
1.	Hardware and Software Requirements	4
2.	Gap Analysis b/w Existing System & User Needs	9
3.	Technology Stack Summary	18
4.	Test Case Table	47
5.	Comparison with Traditional Preparation Methods	50

LIST OF FIGURES

Figure No.	Name of Figure	Page No.
1.	Level 0 DFD	28
2.	Level 1 DFD	29
3.	Level 2 DFD	30
4.	Use Case Diagram	31
5.	Sign Up Page	39
6.	Login Page	39
7.	User Dashboard	41
8.	Interview Creation	42
10.	Coding Practice interface	43
12.	AI And Other Sections	44

CHAPTER 1

INTRODUCTION

1.1 Background of the Problem

In the contemporary digital era, the process of talent acquisition has undergone significant transformation. Organizations across industries—ranging from IT firms to consulting agencies and multinational corporations—rely heavily on structured interviews to assess the technical capabilities, analytical thinking, communication quality, and problem-solving skills of prospective candidates. Although numerous online resources exist, the majority of learners still struggle with effective and structured interview preparation.

The central challenge is rooted in the lack of personalized, immediate, and data-driven feedback. Traditional preparation methods such as reading blogs, watching tutorials, or practicing answers in isolation often fail to simulate a realistic interview environment. Additionally, candidates seldom receive constructive, unbiased, and timely evaluation on their responses. This makes self-evaluation difficult—especially for those who may not have access to mentors or mock interview coaches.

With the rapid advancement of Artificial Intelligence (AI) and Large Language Models (LLMs), modern systems possess the capability to understand, evaluate, and generate humanlike responses. This technology can analyze user inputs, identify strengths and weaknesses, and provide insights comparable to an expert interviewer. Despite this potential, existing platforms do not use AI efficiently to provide holistic interview preparation, leaving a technological gap between available tools and learner requirements.

CodeCraft merges as a solution to bridge this gap by offering an AI-driven interview preparation ecosystem. Leveraging the capabilities of the Gemini API, it delivers real-time evaluation, generates dynamic interview questions, and provides actionable feedback that helps users improve steadily. The system is built using modern web technologies such as Next.js, React, ShadCN UI, Tailwind CSS, and MongoDB, ensuring scalability, responsiveness, security, and user-centric design.

Thus, the project is motivated by the need to create a smart, adaptive, and scalable system that empowers learners to practice interviews anytime, receive instant evaluation, and track their improvement over time.

1.2 Problem Statement

Most existing interview preparation systems rely on static content, predefined question banks, or human-evaluated mock interviews. These approaches present several limitations:

1.2.1 Lack of Dynamic Questioning : Traditional platforms provide fixed sets of questions that become repetitive. They fail to adapt to the user's learning progress or chosen domain.

1.2.2 Inability to Offer Real-Time Evaluation : Human evaluation is time consuming, subjective, and inconsistent. Learners preparing independently often have no way to judge whether their answer meets professional standards.

1.2.3 Limited Personalized Feedback : Most systems offer generic tips rather than targeted insights such as clarity, tone, structure, relevance, depth of explanation, or technical accuracy.

1.2.4 Absence of Performance Tracking Mechanisms : There is no consolidated way to store user responses, monitor improvement, or compare performance over time.

1.2.5 Accessibility Barriers : Not all learners have access to interview coaches or mentors. As a result, preparation becomes unequal and dependent on personal networks.

To address these challenges, the problem can be formally stated as:

“To design and implement an AI-powered interview preparation system that generates dynamic questions, evaluates user responses in real time, and provides detailed, unbiased feedback while ensuring secure user authentication and scalable data management.”

CodeCraft tackles this by integrating AI evaluation with modern web technologies to create an efficient, user-friendly, and intelligent interview preparation environment.

1.3 Aims and Objectives

1.3.1 Aim of the Project : The primary aim is to develop a smart, scalable, AI-driven interview preparation platform that enhances user learning through meaningful interaction, automated assessment, and personalized insights.

1.3.2 Detailed Objectives :

Objective 1: Develop Intelligent Question Generation

- Generate domain-specific and general interview questions using AI.
- Customize difficulty based on user preferences or categories.

Objective 2: Automate Answer Evaluation and Feedback

- Utilize Gemini API to analyse user responses.
- Provide structured feedback on clarity, correctness, communication, and relevance.

Objective 3: Build a User Authentication and Management System □ Use

Clerk for secure login, signup, and identity management.

- Preserve user sessions and protect sensitive data.

Objective 4: Create a Scalable, Responsive Frontend

- Implement an intuitive UI using React, Next.js, Tailwind CSS, and ShadCN UI components.
- Ensure accessibility across devices and screen sizes.

Objective 5: Implement Efficient Data Storage

- Store user responses, performance evaluations, and history using MongoDB.
- Ensure fast retrieval and data integrity.

Objective 6: Offer Real-Time Insights and Analysis

- Present evaluation results in a well-structured format.
- Display improvement metrics and performance graphs (if implemented).

1.4 Scope of the Project

CodeCraft covers a wide range of functionalities aimed at providing a robust and accessible interview preparation platform. Its scope includes:

1.4.1 User-Centric Features

- Create mock interview sessions
- Generate questions dynamically
- Submit and evaluate answers instantly
- View structured feedback generated by AI
- Track history of past attempts

1.4.2 System-Level Functionality

- Secure authentication through Clerk
- Scalable API handling with Next.js

- Cloud-ready storage system through MongoDB
- Session-based data tracking

1.4.3 Technologies and Integrations

- Integration with Gemini API for natural language evaluation
- Use of ShadCN UI for accessible user interface components
- Tailwind CSS for maintainable and responsive design
- Integration-ready design for future expansion

1.4.4 Out-of-Scope Items

- Real-time audio interview simulation
- Behavioural analysis through voice or video
- Resume analysis
- Company-specific interview scheduling

1.5 Hardware and Software Requirements

Component	Minimum Requirement	Recommended
Processor	Intel i3 or equivalent	Intel i5 or higher
RAM	4 GB	8 GB or higher
Storage	500 MB	2 GB for development
Connectivity	Stable Internet	High-speed broadband

1.5.1 Hardware Requirements

1.5.2 Software Requirements

Operating System

- Windows 10 / 11
- macOS
- Linux distributions (Ubuntu preferred)

Development Tools

- Visual Studio Code
- Node.js (LTS version)
- MongoDB
- Git version control
- NPM or Yarn package manager

Libraries & Frameworks

- Next.js
- React.js
- Tailwind CSS
- ShadCN UI
- Clerk Authentication SDK
- Gemini API Client
- Mongoose ORM

1.5.3 Users and Stakeholders

- Students preparing for interviews
- Job seekers
- Professional candidates
- Institutions providing practice platforms
- Recruiters seeking automated mock evaluation tools

CHAPTER 2

INTRODUCTION TO EXISTING SYSTEM

2.1 Overview of Existing Interview Preparation Methods

Interview preparation has traditionally relied on highly fragmented resources. Students and job seekers typically use online blogs, video tutorials, question banks, career guidance articles, or peer discussions to prepare for interviews. These approaches, although helpful, do not offer a structured ecosystem where learning and evaluation function together. The majority of existing methods revolve around self-study, where candidates assess their own performance without any expert feedback. As a result, the learning process becomes largely dependent on subjective judgment.

Many existing digital platforms provide collections of interview questions organized into categories such as technical, behavioural, and HR-based topics. However, these platforms present the same static set of questions to every user. They do not adapt content according to the learner's progress, past performance, or skill level. Furthermore, such websites rarely provide mechanisms for evaluating answers, making it challenging for learners to identify whether they are improving.

In addition to these limitations, traditional mock interviews conducted by institutions or mentors are time-consuming and require the presence of an evaluator. This restricts accessibility, especially for candidates who do not have easy access to mentors or career instructors. Moreover, human evaluators may unintentionally exhibit bias, inconsistency, or variability in their feedback, adding another layer of uncertainty for learners.

These challenges create a clear need for a modern, automated, and intelligent interview preparation system capable of delivering personalized learning experiences and performance based insights to users.

2.2 Existing Web-Based Platforms

A wide range of interview preparation platforms exists today, but most suffer from common shortcomings. These platforms include coding practice websites, online bootcamps, question banks, and mock interview applications. Some of the popular categories of platforms include:

2.2.1 Question-Bank Based Platforms : Websites like GeeksforGeeks, InterviewBit, and LeetCode provide extensive collections of interview questions. While these platforms are excellent for practicing structured technical problems, they do not focus on:

- Analytical explanation-based answers
- Behavioural interviews
- Soft skills assessment
- Real-time evaluation

2.2.2 Video-Based Learning Platforms : Platforms such as YouTube, Udemy, and Coursera offer interview preparation courses. Although informative, these courses follow a one-directional learning model and lack interactive assessments. Learners cannot practice and receive feedback simultaneously.

2.2.3 Human-Based Mock Interview Services : Some platforms provide mock interviews with industry professionals. While these interactions are valuable, they come with limitations:

- High cost
- Scheduling difficulties
- Limited availability
- Subjective feedback
- Slow learning cycle

2.2.4 Chatbot-Based Learning Tools : Certain AI chatbots or Q&A tools allow users to ask interview questions, but:

- They do not evaluate user responses
- They do not store performance history
- They do not generate structured feedback
- They lack a guided mock interview flow

2.3 Limitations of Existing Systems

Regardless of the variety of platforms available, several major limitations continue to affect the quality of interview preparation.

2.3.1 Static and Non-Contextual Question Sets : Most platforms rely on fixed question banks. Learners repeatedly encounter the same questions, leading to memorization rather than skill development. These systems do not generate personalized questions based on user categories, domains, or improvement areas.

2.3.2 Absence of Real-Time Feedback : The most significant gap in current systems is the lack of instant, structured feedback. Without feedback, users cannot determine:

- Whether their answer is correct
- How to improve articulation
- Whether the explanation is technically sufficient
- Whether the response is suitable for interview standards

2.3.3 Lack of Performance Tracking : Existing systems rarely provide:

- Performance graphs
- Historical evaluation
- Comparison of improvement over time
- Category-wise scoring trends

2.3.4 Limited Accessibility and Scalability : Human-based mock interview systems cannot scale to support large numbers of users. They also require paid subscriptions and live scheduling, making them inaccessible to many learners.

2.3.5 Low Personalization : Almost no existing platform adapts its difficulty level or question patterns based on user behaviour. Every learner receives the same content, regardless of prior performance.

2.3.6 Fragmented Learning Experience: Learners often switch between:

- YouTube videos
- Websites
- Notes
- Blogs
- External mentors

2.4 Need for an Intelligent Interview System

A modern interview preparation platform must acknowledge the complexity of both technical and behavioural interviews. Interviews today evaluate:

- Analytical reasoning
- Technical proficiency
- Communication clarity
- Domain-specific understanding
- Confidence and articulation
- Ability to structure thoughts

To cater to these multidimensional evaluation parameters, a system must:

1. Be adaptive : Questions should adjust based on the learner's interests, category selection, and performance history.

2. Offer instant feedback : Candidates need immediate insights into their strengths and weaknesses to improve rapidly.
3. Provide structured analysis : Feedback should highlight:
 - Accuracy
 - Depth of explanation
 - Language usage
 - Missing points
 - Suggestions for improvement
4. Store performance history : Users should be able to monitor trends and growth over time.
5. Be available 24/7 : AI-based evaluation ensures that learners can practice anytime.
6. Ensure secure user authentication : User data must be protected.

2.5 Gap Analysis Between Existing Systems and User Needs

User Requirements	Existing Systems	Identified Gaps
Dynamic question generation	Mostly static banks	No personalization
Automated evaluation	Rarely available	No AI-based scoring
Structured feedback	Not offered	Lack of actionable insights
Performance tracking	Minimal	No long-term analytics

CHAPTER 3

PROPOSED SYSTEM

3.1 Overview of the Proposed System

The proposed system, Pre2Place, is designed as a fully automated, AI-powered interview preparation platform that bridges the gap between traditional static preparation methods and the evolving demands of modern recruitment processes. The system combines intelligent question generation, AI-driven evaluation, secure authentication, and structured analysis within a centralized environment. Unlike existing solutions, Pre2Place focuses on adaptability, personalisation, and continuous feedback.

Through the integration of Gemini API, the system evaluates user responses with high accuracy by analysing depth, correctness, clarity, structure, and relevance. This ensures the feedback resembles the insights provided by a professional interviewer. The frontend is developed using modern technologies such as Next.js, React, Tailwind CSS, providing a seamless user experience with responsive performance across devices. User authentication is managed using Clerk, which ensures secure, reliable, and scalable user identity management. The backend system stores session data, answers, and performance metrics in MongoDB, allowing efficient retrieval and long-term tracking.

This holistic architecture enables CodeCraft to deliver instant, intelligent, and personalized learning to candidates preparing for interviews.

3.2 Core Idea Behind the System

The idea behind Pre2Place revolves around creating a platform that functions like a virtual interview mentor. The system should emulate the experience of a real interview — presenting questions, assessing responses, and offering constructive feedback instantly.

A user-centric approach was prioritized, ensuring that:

- Questions match the user's selected category or domain
- Answers are evaluated contextually rather than generically
- Feedback highlights strengths, weaknesses, and suggestions
- Sessions are stored for progression tracking
- Users can practice at their own pace, anytime

3.3 Detailed Features of the Proposed System

3.3.1 Intelligent Question Generation: CodeCraft generates interview questions dynamically using Gemini API and predefined categories. Unlike static question banks, generated questions are:

- Relevant to the user's chosen topic
- Diverse and non-repetitive
- Aligned with professional interview standards
- Suitable for both beginners and advanced users

3.3.2 AI-Based Answer Evaluation : The system analyses user-submitted answers in real time, evaluating them on parameters such as:

- Technical correctness
- Completeness of explanation
- Clarity and fluency
- Structured communication
- Depth of understanding
- Relevance to the question

The Gemini API generates detailed feedback that includes:

- What was correct in the answer
- What was missing
- Suggestions for improvement
- Alternative phrasing for better structure

3.3.3 Secure User Authentication : Using Clerk, CodeCraft provides:

- User login and sign-up
- OAuth options (Google, GitHub, etc.)
- Session protection
- Logout management
- Profile security

3.3.4 User Dashboard and UI Flow : The interface is developed using:

- Next.js for server-side rendering and optimized routing
- React for modular, component-based UI
- Tailwind CSS for responsive design
- ShadCN UI for consistent, professional-looking components

The dashboard includes:

- Category selection
- Question generation panel
- Answer submission area
- Feedback display area
- History and session management options

3.3.5 Session Management : The system stores:

- Questions generated
- User answers
- AI feedback
- Date and time of practice session

3.3.6 Performance Tracking : Although basic in the current version, future versions include or plan to include:

- Score trends over time
- Graphical performance improvement
- Category-wise strengths and weaknesses

3.3.7 Scalable Backend Architecture : The backend uses:

- Next.js API routes for handling requests
- MongoDB for storing evaluation data
- Mongoose for structured schemas and validation

3.4 System Objectives Fulfilled by the Proposed Solution

The proposed system satisfies multiple objectives required for an efficient interview preparation platform:

Objective 1: Automate Interview Practice - CodeCraft eliminates the need for human evaluators by creating a fully virtual mock interview experience empowered by AI.

Objective 2: Provide Personalized Feedback - The system does not rely on generic tips. Instead, it tailors feedback to each user's unique answer.

Objective 3: Improve Accessibility - Users can practice anytime, anywhere, eliminating scheduling barriers associated with human mock interviews.

Objective 4: Enhance Learning Speed - Immediate feedback accelerates the learning cycle significantly compared to traditional preparation.

Objective 5: Maintain Data Security - Clerk ensures that user identities and session data remain secure.

3.5 Uniqueness of the Proposed System

CodeCraft stands out because:

1. It uses generative AI for contextual evaluation.
2. It provides structured and understandable feedback.
3. It is built using modern full-stack technologies.
4. It improves with every user interaction.
5. It focuses on both technical knowledge & communication.

3.6 Advantages of the Proposed System

- 3.6.1 Continuous Learning Support : Users can practice without waiting for mentors.
- 3.6.2 Reduces Anxiety : AI-generated feedback helps reduce fear by facilitating regular practice.
- 3.6.3 Objective Evaluation : The system removes biases common in human evaluation.
- 3.6.4 Cost-Effective : Unlike paid mock interview services, CodeCraft is free or low cost depending on deployment.
- 3.6.5 Scalable for Institutions : Colleges can implement the system for large groups without additional human resources.
- 3.6.6 Improves Communication Skills : Feedback highlights not just technical accuracy but also communication clarity.

3.7 Summary

The proposed system fulfils the need for an intelligent, accessible, and unbiased interview preparation platform. By combining AI-based evaluation, modern user interface principles, secure authentication, and scalable backend architecture, CodeCraft offers a comprehensive solution to common challenges faced by interview candidates. It bridges the gap between traditional preparation and modern industry standards.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Introduction to System Architecture

System architecture represents the structural design of a software system, depicting how various components interact, communicate, and work collectively to fulfil system objectives. In any web-based application, especially one leveraging AI-driven evaluation mechanisms, architecture plays a crucial role in ensuring scalability, accuracy, modularity, and reliability.

The architecture of CodeCraft is centred around a client–server model integrated with an AI-powered evaluation engine using Gemini API. The system is designed to be modular so that each component — authentication, question generation, evaluation, database interaction, and UI rendering — operates independently yet cohesively. This ensures that failures in one module do not affect the entire platform.

The system architecture embraces principles like:

- Separation of concerns
- Loose coupling
- High cohesion
- API-driven communication
- Scalability through cloud-friendly design

4.2 High-Level Architecture Overview

CodeCraft’s architecture consists of the following major components:

1. Frontend Application (UI Layer)
2. Authentication Layer (Clerk)
3. Backend Server / API Routes (Next.js Server Functions)
4. AI Evaluation Engine (Gemini API)
5. Database Layer (MongoDB)

Each component is responsible for specific tasks and communicates with others through clearly defined interfaces.

Description of the High-Level Architecture Flow (Text Diagram Explanation):

User → Frontend (Next.js/React) → Authentication (Clerk) ↓
Backend API Routes → Gemini API (Evaluation Engine) ↓
MongoDB Database → Returns Result → Frontend UI

This flow ensures that:

- The user interacts with a clean UI
- All access is secured via Clerk
- Business logic runs on server-side API routes
- AI evaluates responses
- Data is stored and retrieved efficiently

4.3 Key Architectural Layers

4.3.1 Presentation Layer (Frontend UI) : This layer handles user interaction and is built using:

- Next.js 14 App Router
- React functional components
- Tailwind CSS for styling
- Shad CN UI for modular, reusable components

Key Responsibilities:

- Render UI screens (dashboard, question page, evaluation page, history page)
 - Capture user inputs (answers, selected categories)
 - Send requests to backend server
 - Display AI-generated feedback
 - Fetch and show previous sessions
- Why this architecture?
- Server-side rendering in Next.js improves performance
 - React enables the creation of dynamic, interactive components
 - Tailwind CSS ensures responsive design without heavy CSS files

4.3.2 Authentication Layer (Clerk) : Authentication is a critical component ensuring secure interaction with user-specific data.

Key Responsibilities:

- User signup and login
- OAuth integration (Google, GitHub, Email)
- Session management
- Token generation and validation
- Restricting unauthorized data access

Why Clerk?

Clerk provides robust identity management without requiring custom backend logic, reducing development complexity while maximizing security.

4.3.3 Backend Logic Layer (Next.js API Routes) : This layer functions as the intermediary between the frontend and the AI engine or database.

Key Responsibilities:

- Handle API requests
- Generate interview questions (using static prompts or AI calls)
- Process user-submitted answers
- Call Gemini API for evaluation
- Connect to MongoDB for storing session data
- Format responses for the frontend UI

Advantages of Using Next.js for Backend:

- Serverless deployment capabilities
- Built-in routing
- Native API handling without additional frameworks
- Reduced latency and enhanced performance

4.3.4 AI Evaluation Layer (Gemini API) : This is the intelligent core of the system where actual evaluation happens.

Key Responsibilities:

- Receive user answer
 - Understand question context
 - Evaluate accuracy, clarity, relevance, and completeness
 - Generate structured feedback
 - Return evaluation for UI display and storage
- How Gemini API Works Here:
- CodeCraft forms a prompt containing the question + user's answer
 - Gemini processes it using its LLM reasoning capability
 - Output is a detailed, structured feedback string
- Why Gemini API?
- High linguistic understanding
 - Context sensitivity
 - Ability to structure long-form evaluations
 - Accurate summarization capabilities

4.3.5 Database Layer (MongoDB) : MongoDB stores user-specific data as collections and documents.

Data Stored Includes:

- User ID
- Practice session records
- Generated questions
- User answers
- AI feedback
- Timestamp logs

Why MongoDB?

- Flexible schema for storing JSON-like documents
- High-speed read/write operations
- Easy integration with Next.js
- Cloud scalability

4.4 Detailed Module-Level Architecture

4.4.1 User Module

- Profile management
- Session fetching
- Navigation between UI screens

4.4.2 Question Generation Module

- Selecting category
- Generating random or AI-based questions
- Serving them to the UI

4.4.3 Answer Submission Module

- Capturing user answers □ Validating input format
- Sending answers to API for processing

4.4.4 Evaluation Module

- Construct evaluation prompts
- Receive detailed feedback
- Parse data
- Format final evaluation object

4.4.5 Storage Module

- Storing evaluation results
- Fetching user history
- Syncing with authentication token

4.5 Data Flow Description

Below is the verbal representation of Data Flow :

Step-by-Step Data Movement:

1. User logs in → Clerk validates credentials
2. User selects a category → Frontend sends request to Backend
3. Backend generates or fetches a question
4. Question displayed to user
5. User submits answer → Frontend → Backend
6. Backend sends prompt to Gemini API
7. Gemini API returns evaluation
8. Backend stores evaluation in MongoDB
9. Frontend retrieves evaluation and displays it
10. User can later view history through backend queries

This flow ensures a smooth, continuous interaction loop between frontend, backend, AI model, and database.

4.6 Architectural Design Principles Used

1. Modularity: Each component performs a single, well-defined task.
2. Scalability : Architecture supports future expansion such as voice interviews, coding evaluations, or resume analysis.
3. Security : Authentication and token-based authorization prevent unauthorized access.
4. High Availability: Frontend and backend can be deployed on separate servers or serverless platforms like Vercel.
5. Maintainability : Code components are separated into logical folders, making the project easy to maintain.

4.7 Technology Stack Justification

1. Next.js : Chosen for SEO optimization, performance, server-side capabilities, and smooth routing.

2. React : Used for dynamic UI behavior and reusable components.
3. Tailwind CSS : Reduces CSS complexity and ensures a consistent design language.
4. Shad CN UI : Provides clean, professional, production-ready prebuilt components.
5. Clerk: Modern authentication system that reduces backend workload.
6. MongoDB : Flexible database that handles document-based storage effectively.
7. Gemini API : Advanced LLM providing strong contextual evaluation for interview answers.

4.8 Summary

The architecture of CodeCraft is designed to be modern, modular, efficient, and AI-driven. By leveraging Next.js for frontend and backend integration, Clerk for secure authentication, MongoDB for powerful storage, and Gemini API for intelligent evaluation, the system offers a scalable and robust solution for interview preparation. The architecture ensures seamless communication between all components, delivering a reliable and interactive user experience.

CHAPTER 5

TECHNOLOGY STACK

5.1 Introduction to Technology Stack

A technology stack refers to the combination of programming languages, frameworks, libraries, tools, and services that together enable the development, execution, and deployment of a software application. The effectiveness of a system depends greatly on choosing the right technologies that support scalability, performance, security, and future enhancements.

CodeCraft uses a modern, full-stack, cloud-friendly, AI-integrated technology stack that ensures optimal user experience and robust backend performance. This chapter discusses all major technologies used in the application and the rationale behind selecting each.

5.2 Frontend Technologies

5.2.1 Next.js : Next.js is the foundational framework used to build CodeCraft's frontend. It offers both server-side rendering (SSR) and client-side rendering (CSR) capabilities, enabling highly performant and SEO-friendly applications.

Reasons for Choosing Next.js:

- Hybrid architecture: SSR + CSR
- Built-in routing system
- Better performance through pre-rendering
- API route support for backend logic
- Highly scalable and Vercel-optimized ☐ Image and asset optimization

Role in CodeCraft:

- Renders UI pages efficiently
- Handles routing between dashboard, interview pages, history pages
- Supports backend API endpoints under the same framework
- Improves user experience through fast loading screens

5.2.2 React.js : React powers the component structure of the UI. It supports a modular architecture, making the interface easier to maintain and scale.

Why React?

- Component-based design
- Reusability of code
- Virtual DOM boosting performance
- Large community and ecosystem
- Easy state management

Use in CodeCraft:

- UI components like buttons, cards, modals, question panels
- State management for user inputs
- Dynamic rendering of evaluation results

5.2.3 Tailwind CSS : Tailwind CSS is a utility-first CSS framework enabling rapid UI development with minimal custom CSS.

Advantages:

- Highly customizable design
- Eliminates the need for writing long CSS files
- Light-weight and optimized
- Excellent responsive design capabilities

Use in CodeCraft:

- Designing responsive UI layouts
- Styling forms, containers, buttons, and feedback components
- Maintaining visual consistency across pages

Prepared UI Components in CodeCraft:

- Cards
- Text areas
- Inputs
- Buttons
- Sheets and modals

5.3 Backend Technologies

5.3.1 Next.js API Routes : Instead of using a separate backend framework like Express, CodeCraft uses Next.js API Routes, enabling both frontend and backend to coexist within a single codebase.

Benefits:

- Less boilerplate
- Easy integration with frontend
- Reduced latency
- Serverless architecture-friendly
- Best suited for small-to-medium backend operations

Backend Tasks Managed:

- Call Gemini API
- Process user answers
- Communicate with MongoDB
- Store and retrieve session data

5.3.2 Node.js : Node.js acts as the runtime environment allowing JavaScript to run on the server.

Why Node.js?

- Non-blocking I/O for efficiency
- Vast ecosystem via NPM
- Fast execution using V8 engine

Role in CodeCraft:

- Handles asynchronous operations such as API calls
- Supports server-side functions
- Defines backend logic within Next.js routes

5.4 Authentication Technology

5.4.1 Clerk Authentication : Clerk is a cutting-edge identity management solution providing highly secure authentication and user account handling.

Key Features:

- Multi-provider OAuth
- Prebuilt UI widgets for login/signup
- Session token management
- JWT validation
- Access control
- User profile storage

Why Clerk Over Firebase Auth / Auth0?

- Easier integration with Next.js

- Better documentation
- Cleaner UI prebuilt components
- No manual token refresh logic
- Scalable authentication handling

Role in CodeCraft:

- Ensures only verified users access the mock interview tools
- Protects user-specific data and history
- Handles login, logout, session state

5.5 Artificial Intelligence Technology

5.5.1 Gemini API : Gemini API by Google is the Large Language Model (LLM) integrated into CodeCraft to evaluate user answers.

Capabilities of Gemini API:

- Natural language understanding
- Context awareness
- Ability to evaluate structured/unstructured answers
- Generate detailed feedback
- Offer improvement suggestions □ High reasoning ability

Why Gemini API?

- More accurate contextual evaluation
- Faster inference time
- Consistent and unbiased feedback □ Professional answer formatting

Role in CodeCraft:

- Evaluates user answers
- Scores clarity, correctness, and completeness
- Generates improvement recommendations
- Acts as the core intelligence of the system

5.6 Database Technology

5.6.1 MongoDB : MongoDB is used as the primary database for storing all user information and interview session details.

Key Features:

- Schema flexibility

- Fast read/write performance
- JSON-based document structure
- Cloud compatibility (MongoDB Atlas)
- Horizontal scalability

Data Stored in MongoDB:

- User ID
- Generated questions
- User answers
- AI evaluations
- Session timestamps

5.7 Additional Tools and Libraries

5.7.1 Mongoose : A MongoDB Object Data Modelling (ODM) library.

Role:

- Defines schemas
- Validates data
- Simplifies CRUD operations

5.7.2 Axios or Fetch API : Used to send API requests between frontend and backend.

Purpose:

- Send answer data
- Receive Gemini feedback
- Fetch session history

5.7.3 Vercel Deployment : Vercel provides hosting for:

- Frontend
- Backend API routes

Benefits:

- Zero-config deployments
- Serverless optimization
- Automatic scaling

5.8 Technology Stack Summary

Layer	Technology Used	Purpose
-------	-----------------	---------

Frontend	Next.js, React, Tailwind CSS	UI rendering, routing, styling
Authentication	Clerk	Secure sign-in, session handling
Backend	Next.js API Routes, Node.js	Business logic, request handling
AI Engine	Gemini API	Evaluation, feedback generation
Database	MongoDB, Mongoose	Data storage and retrieval

CHAPTER 6

SYSTEM DESIGN

6.1 Introduction to System Design

System design defines how a software system is planned, structured, and organized before implementation. It acts as a blueprint that translates requirements into a technical solution. A well-designed system improves scalability, maintainability, security, and performance.

For CodeCraft, system design focuses on creating a modular, AI-driven, and user-centric platform that ensures smooth interaction between users, backend services, artificial intelligence components, and the database. The design emphasizes clarity, separation of concerns, and efficient data flow.

The system design of CodeCraft is divided into:

- Functional design
- Structural design
- Behavioural design
- Data design

6.2 Functional Design of the System

Functional design explains what the system does and how each function supports the overall objective.

Major Functionalities of CodeCraft

- User registration and authentication
- Category-based interview selection
- Dynamic question generation
- Answer submission and validation
- AI-based evaluation and feedback generation
- Storage of interview sessions
- Retrieval of previous attempts □ Secure access control

6.3 Use Case Description

6.3.1 Overview of Use Case Model : A use case model describes how users interact with the system. It focuses on user goals rather than internal system structure.

CodeCraft primarily involves one main actor: User.

Primary Actor

- User: Any registered individual using the platform to practice interviews.

Secondary Systems

- Authentication service
- AI evaluation engine
- Database system

6.3.2 Major Use Cases

Use Case 1: User Registration

- Actor: User
- Description: The user creates a new account using Clerk authentication.
- Outcome: Secure user profile is created.

Use Case 2: User Login

- Actor: User
- Description: The user logs into the system using valid credentials.
- Outcome: User session is established.

Use Case 3: Select Interview Category

- Actor: User
- Description: The user selects a domain or category for interview practice.
- Outcome: System prepares relevant question prompts.

Use Case 4: Generate Interview Question

- Actor: System
- Description: The system generates a dynamic interview question using predefined logic or AI prompts.
- Outcome: Question displayed to the user.

Use Case 5: Submit Answer

- Actor: User

- Description: The user submits an answer to the displayed question.
- Outcome: Answer sent to backend for evaluation.

Use Case 6: AI Evaluation

- Actor: System
- Description: The system sends question and answer to the AI engine for analysis.
- Outcome: Structured feedback generated.

Use Case 7: View Feedback

- Actor: User
- Description: The user views evaluation results and improvement suggestions.
- Outcome: Learning insights gained.

Use Case 8: View History

- Actor: User
- Description: The user views previous interview sessions.
- Outcome: Progress tracking.

6.4 Data Flow Description (DFD Explanation)

6.4.1 Level 0 Data Flow (Context Level)

At the highest level, CodeCraft is represented as a single process interacting with external entities.

- User provides input (login details, answers)
- System processes the input
- System returns output (questions, feedback, history)



Fig. 1 – DFD Level 0.

6.4.2 Level 1 Data Flow

At this level, the system is divided into multiple processes:

- Authentication Process
- Question Generation Process
- Answer Submission Process
- Evaluation Process □ Data Storage Process

Flow Description:

- User logs in → Authentication module validates user
- User selects category → Question generator responds
- User submits answer → Evaluation module processes it
- Feedback is stored → Displayed to user

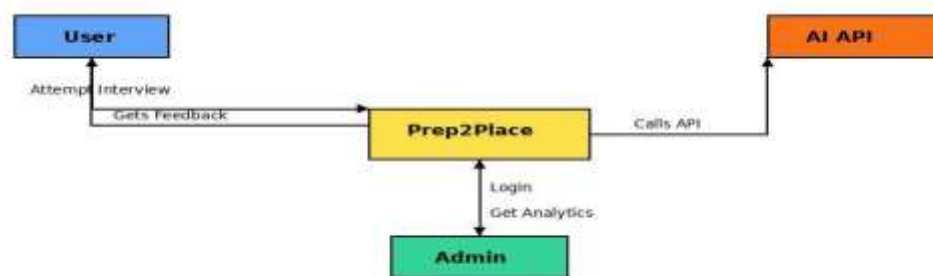


Fig. 2 – DFD Level 1.

6.4.3 Level 2 Data Flow

This level further breaks down individual processes:

- Evaluation Process:
 - Accept answer
 - Construct AI prompt
 - Send request to AI engine
 - Receive response
 - Format feedback
 - Store result



Fig. 3 – Level 2 DFD.

6.5 Entity Relationship Design (ERD Explanation)

6.5.1 Entities Identified

The system identifies the following main entities:

User

- User ID
- Name □ Email
- Authentication ID

Interview Session

- Session ID
- User ID
- Category
- Date
- Time

Question

- Question ID
- Session ID
- Question Text

Answer

- Answer ID
- Question ID
- Answer Text

Evaluation

- Evaluation ID
- Answer ID
- Feedback Text
- Score (optional)

6.5.2 Relationships Between Entities

- One User can have many Interview Sessions
- One Interview Session can have many Questions
- One Question has one Answer
- One Answer has one Evaluation

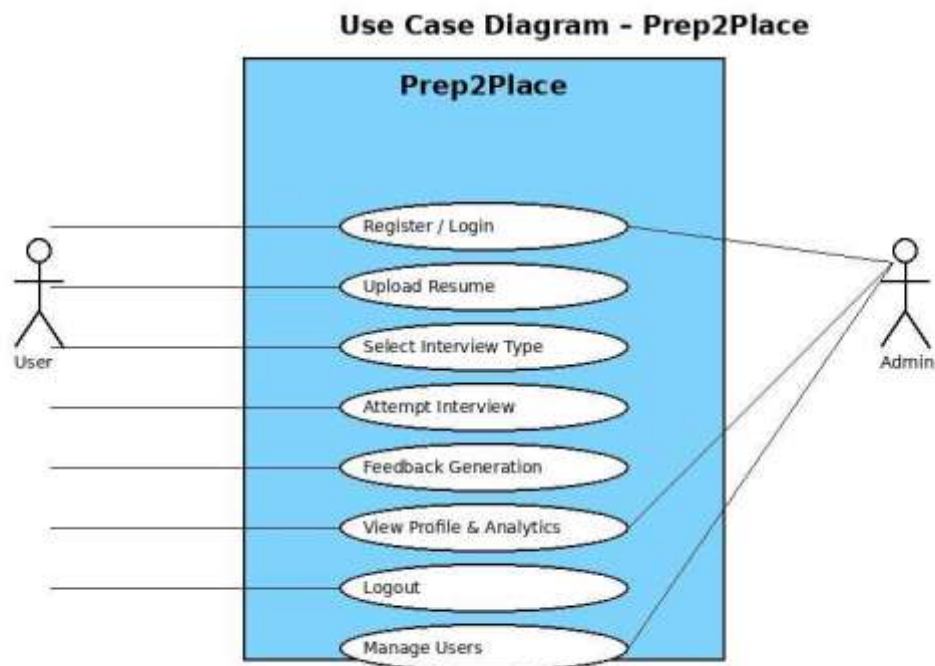


Fig. 4 – Use Case Diagram.

6.6 Database Design Considerations

The database is designed to be:

- Flexible
- Scalable
- Efficient
- Secure

6.7 UML Design Explanation

6.7.1 Class Diagram Description : The class structure includes:

- User Class
- Session Class
- Question Class
- Answer Class
- Evaluation Class

6.7.2 Sequence Diagram Description : Sequence flow:

- User logs in
- System authenticates user
- User selects category
- System generates question
- User submits answer
- System evaluates answer
- Feedback displayed

6.7.3 Activity Diagram Description : Activities include:

- Login
- Category selection
- Question display
- Answer input
- Evaluation
- Feedback viewing
- Session completion

6.8 Input Design

Inputs include:

- Login credentials
- Category selection
- Text-based interview answers

Input validation ensures:

- No empty submissions
- Safe text input
- Valid session handling

6.9 Output Design

Outputs include:

- Generated questions
- AI-generated feedback
- Session history
- Output design focuses on clarity, readability, and usefulness.

6.10 Security Design

Security is implemented through:

- Token-based authentication
- Secure session handling
- Protected API routes
- Database access control

6.11 Summary

The system design of CodeCraft ensures a structured, secure, and scalable platform. By defining clear data flow, entity relationships, functional modules, and behavioural models, the design lays a strong foundation for implementation. This chapter ensures that the system is logically sound, easy to maintain, and capable of supporting future enhancements.

CHAPTER 7

IMPLEMENTATION

7.1 Introduction to Implementation

Implementation is the phase where the conceptual system design is transformed into a working software product. In this phase, all system components such as frontend interfaces, backend services, authentication mechanisms, database interactions, and AI integrations are developed, tested, and integrated.

CodeCraft is implemented using a modern full-stack architecture where both frontend and backend are handled within a single Next.js application. The system follows a modular approach, ensuring that each component can be independently developed, tested, and maintained. Emphasis is placed on clean architecture, reusable components, secure authentication, and efficient data handling.

7.2 Frontend Implementation

7.2.1 User Interface Structure : The frontend of CodeCraft is developed using Next.js with React, following a component-based architecture. Each page and feature is divided into reusable components, ensuring better maintainability and scalability.

The major UI components include:

- Authentication screens (login, signup)
- Dashboard page
- Interview category selection page
- Question display section
- Answer input section
- Evaluation and feedback display section
- History and session review page

7.2.2 Component-Based Design : React components are designed to handle specific functionalities such as:

- Question card rendering
- Answer text input handling
- Submission buttons
- Feedback rendering blocks

- Loading and error states

7.2.3 Styling and Responsiveness : Styling is implemented using Tailwind CSS, which enables utility-first design. This ensures:

- Consistent spacing and layout
- Responsive design across devices
- Clean and professional UI appearance

7.2.4 State Management : State management in CodeCraft is handled using React's built-in hooks:

- Local component state for form inputs
- Loading state during API calls
- Response state for feedback and evaluation

7.3 Authentication Implementation

7.3.1 User Authentication Using Clerk : Authentication is implemented using Clerk, which handles user identity management securely. Clerk provides prebuilt UI components and APIs for:

- User registration
- Login
- Session validation
- Logout

7.3.2 Route Protection : Protected routes are implemented to ensure that:

- Only authenticated users can access interview pages
- Unauthorized access is restricted □ User-specific data is protected

7.4 Backend Implementation

7.4.1 Backend Architecture : The backend is implemented using Next.js API routes, which act as server-side endpoints. These API routes manage:

- Interview question generation
- Answer evaluation requests
- Database storage and retrieval
- Communication with Gemini API

7.4.2 API Request Handling : API routes are designed to:

- Accept HTTP requests from frontend
- Validate incoming data
- Process business logic
- Return structured responses

7.4.3 Error Handling : Error handling mechanisms ensure:

- Graceful failure handling
- Meaningful error messages
- Logging of unexpected errors

7.5 AI Integration Implementation

7.5.1 Gemini API Integration : The Gemini API is integrated to perform answer evaluation. The backend constructs a prompt that includes:

- The interview question
- The user's answer
- Evaluation criteria

7.5.2 Feedback Processing : The AI response is processed to:

- Extract meaningful feedback
- Organize suggestions
- Ensure readability

7.5.3 Performance Considerations : To maintain performance:

- API calls are optimized
- Response sizes are controlled
- Unnecessary requests are avoided

7.6 Database Implementation

7.6.1 Database Connection : MongoDB is used as the primary database. The connection is managed using environment variables to protect sensitive credentials.

7.6.2 Schema Design : Schemas are designed using Mongoose, defining:

- User schema
- Interview session schema
- Question schema
- Answer schema
- Evaluation schema

7.6.3 Data Storage Process : When a user completes a question:

- The answer is received by the backend
- AI feedback is generated
- Both answer and feedback are stored □ Timestamp and user ID are attached

7.7 Implementation of Session History

Session history allows users to:

- Review previous answers
- Read AI feedback
- Analyse improvement areas

7.8 Security Implementation

Security measures include:

- Secure environment variables
- Token-based authentication
- API route protection
- Input validation
- Secure database access

7.9 Challenges Faced During Implementation

7.9.1 AI Response Consistency : Ensuring consistent AI feedback formatting required prompt tuning and response validation.

7.9.2 Managing Asynchronous Calls : Handling multiple asynchronous operations such as API calls and database writes required careful sequencing.

7.9.3 UI Performance Optimization : Ensuring smooth UI performance while handling large text responses required efficient state updates.

7.10 Summary

The implementation phase successfully translates the system design into a functional application. By integrating frontend interfaces, backend logic, authentication services, AI evaluation, and database management, CodeCraft delivers a cohesive and intelligent interview preparation platform. This chapter demonstrates the practical feasibility and robustness of the proposed system.

CHAPTER 8

APPLICATION DEMONSTRATION

8.1 Introduction

This chapter presents a step-by-step demonstration of the CodeCraft application from the user's perspective. It explains how a user interacts with the system, starting from authentication to interview practice and evaluation. The demonstration highlights the functional workflow of the system without exposing internal code details.

The objective of this chapter is to provide a clear understanding of how the system behaves in real-world usage and how different modules integrate seamlessly to deliver an AI-powered interview preparation experience.

8.2 User Authentication Interface

8.2.1 Sign-Up Page : The application begins with a secure authentication interface.. New users are required to register using a valid email address or third-party authentication providers.

Key functionalities demonstrated:

- User registration form
- Input validation
- Secure account creation
- Session initialization

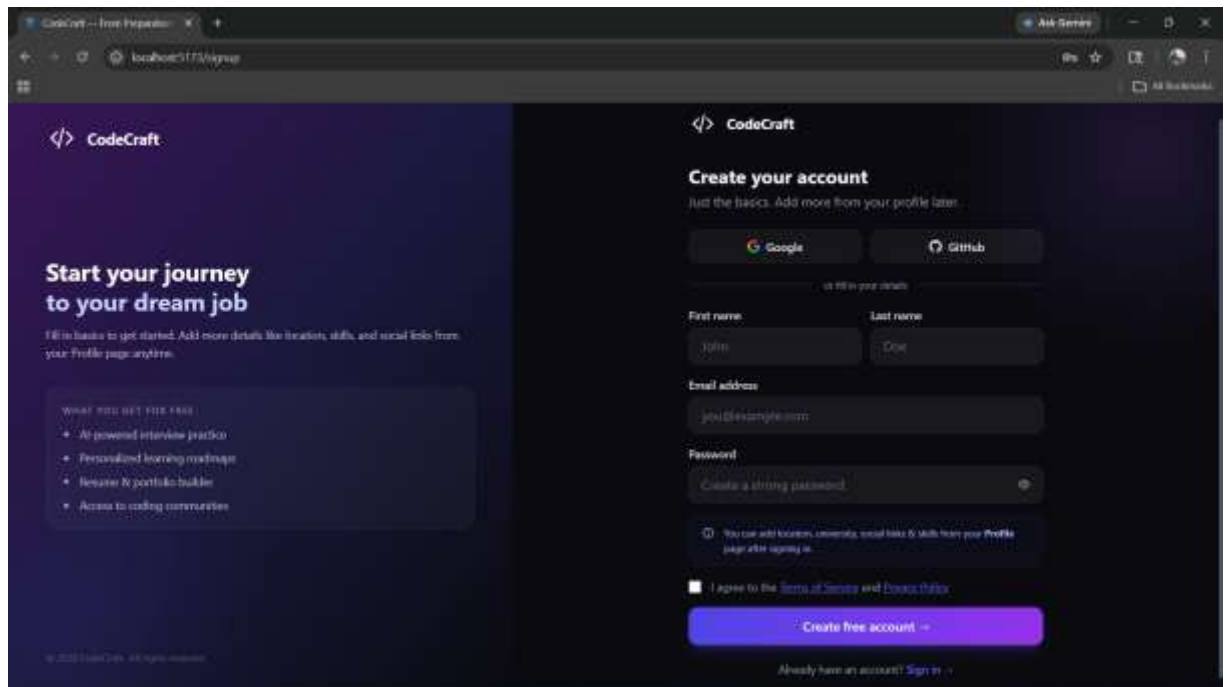


Fig. 5- Sign Up Page of CodeCraft

8.2.2 Login Page : Existing users can log in using their registered credentials. Upon successful login, the system establishes a secure session and redirects the user to the main dashboard.

Purpose of this step:

- Identity verification
- Session creation
- Secure access to user-specific data

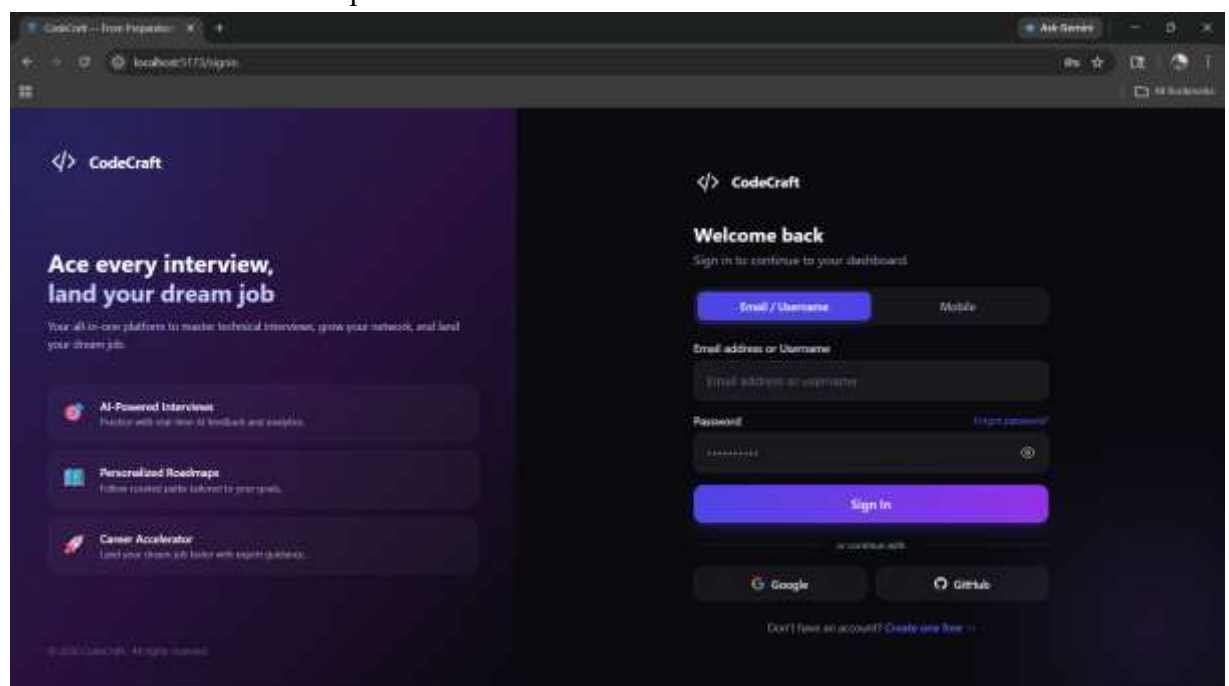


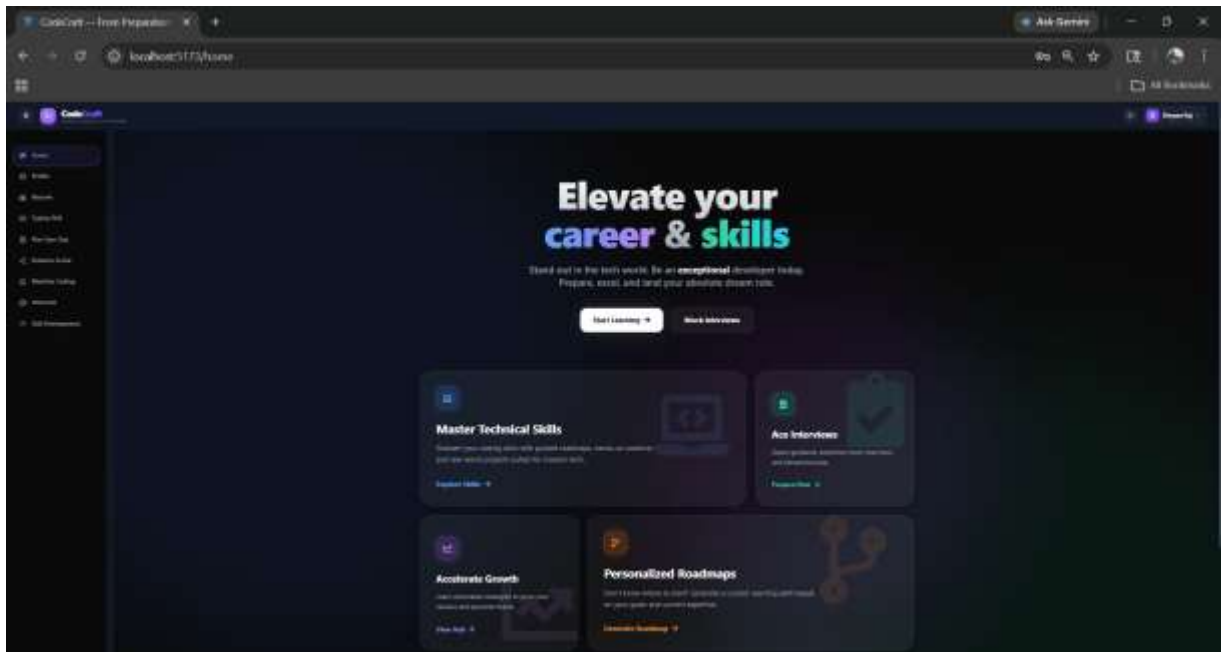
Fig. 6- Login Page of CodeCraft

8.3 User Dashboard

8.3.1 Dashboard Overview : After successful login, the user is redirected to the dashboard. The dashboard acts as the central navigation hub of the application.

Displayed elements include:

- Welcome message
- Option to start a new interview session
- Access to previous interview history
- Logout option



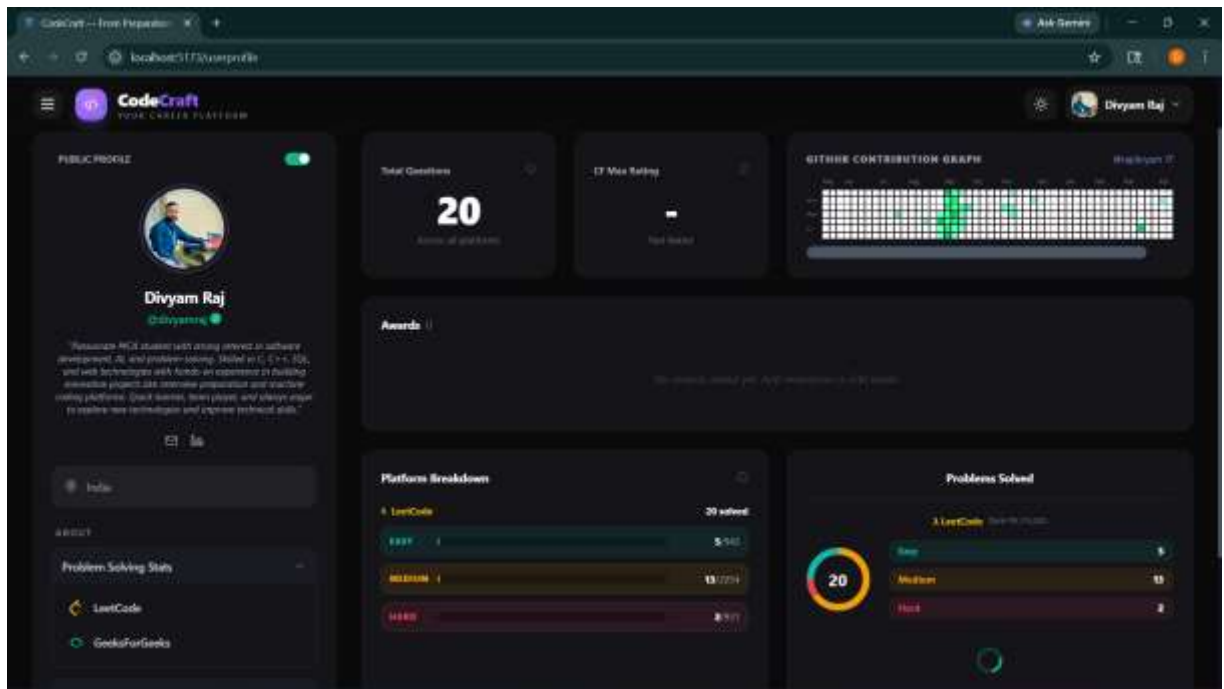


Fig. 7- User Dashboard of CodeCraft .

8.4 Profile Interview Selection Screen

8.4.1 Category Selection : The user selects an interview category or domain from the available options. This selection determines the nature and context of the interview questions generated.

Examples of categories:

- SDE-1 Preparations
- SDE-2 Preparations
- Data Analyst Preparations

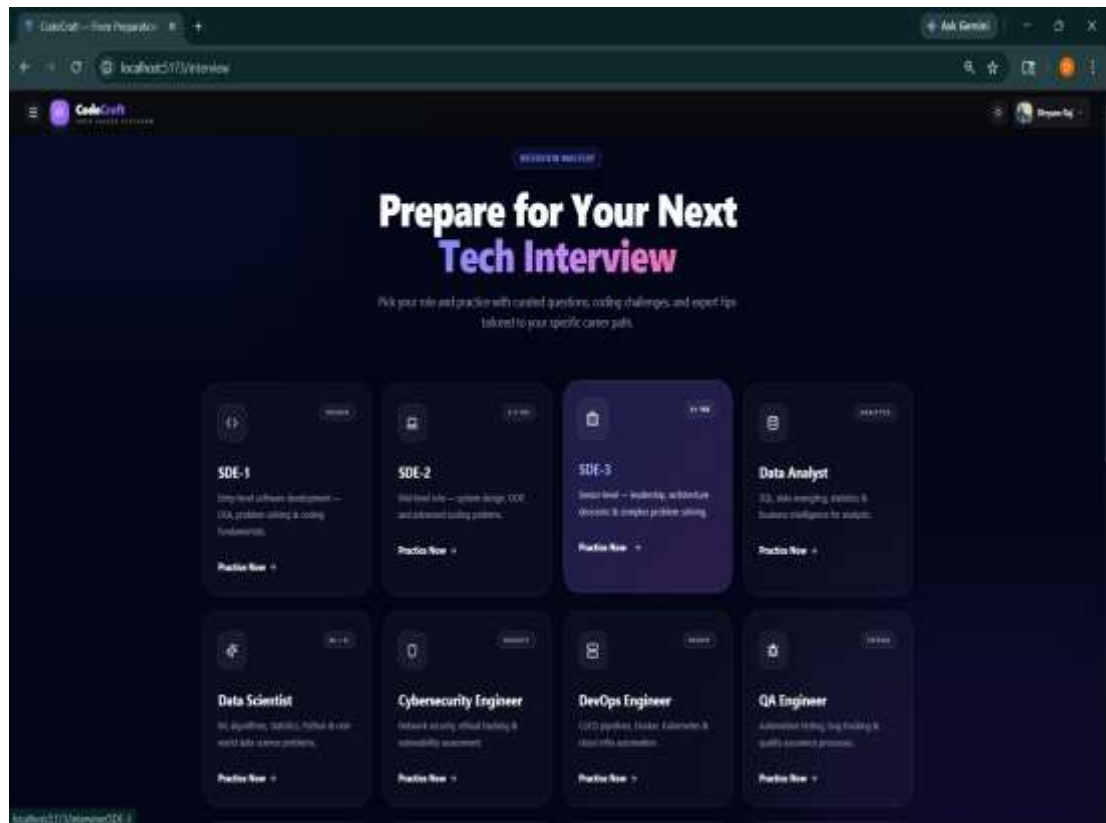


Fig 8. – Interview Creation.

8.5 Coding Practice interface

8.6.1 Code Editor Section : The user is provided with code editor to practice coding questions.

Key interface features:

- Option to opt for programming language like Java , C, Python Submit button
- Output interface where user can see output of its logic
- Pre load Questions to practice

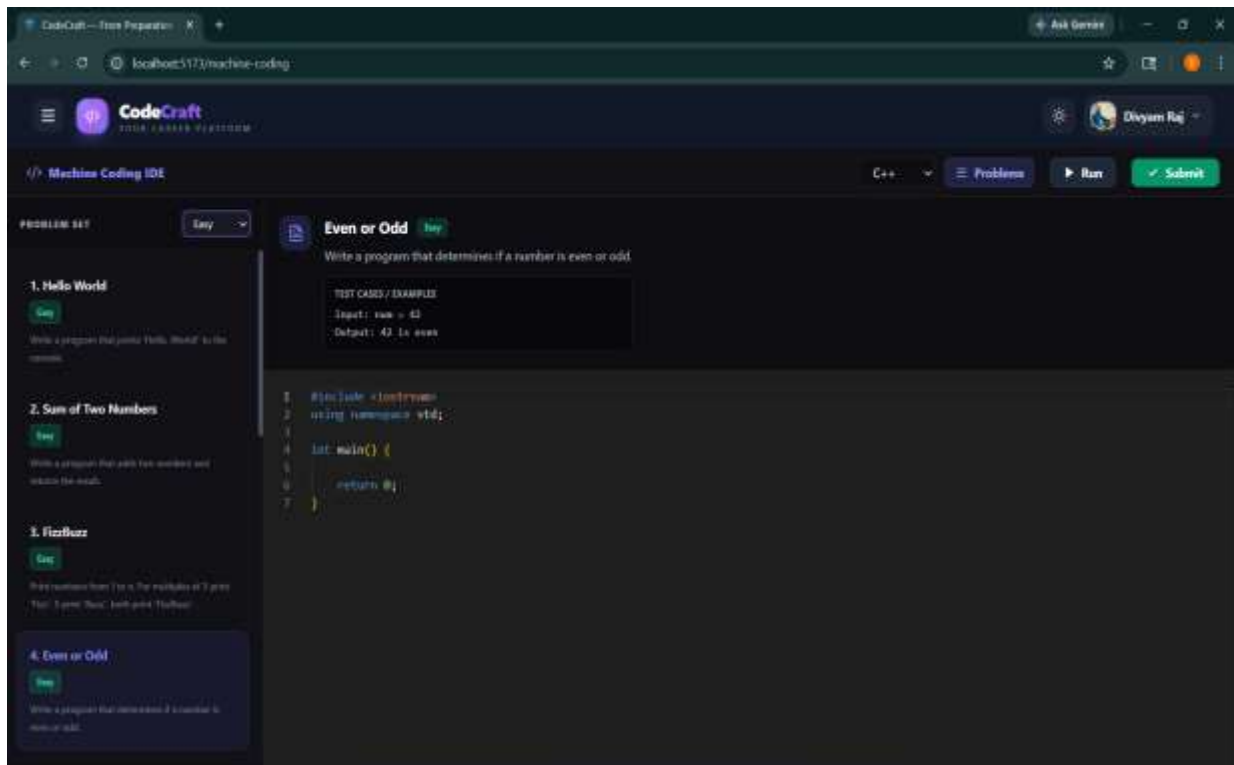


Fig 10. – Coding Practice interface.

8.6 AI Chatbot and other Sections

8.8.1 Choose Ai to resolve query like gemini : Users can opt for AI to solve their query to get the answer.

Features included:

- Ask question
- Upload file
- Type your query
- Create Events
- Join community

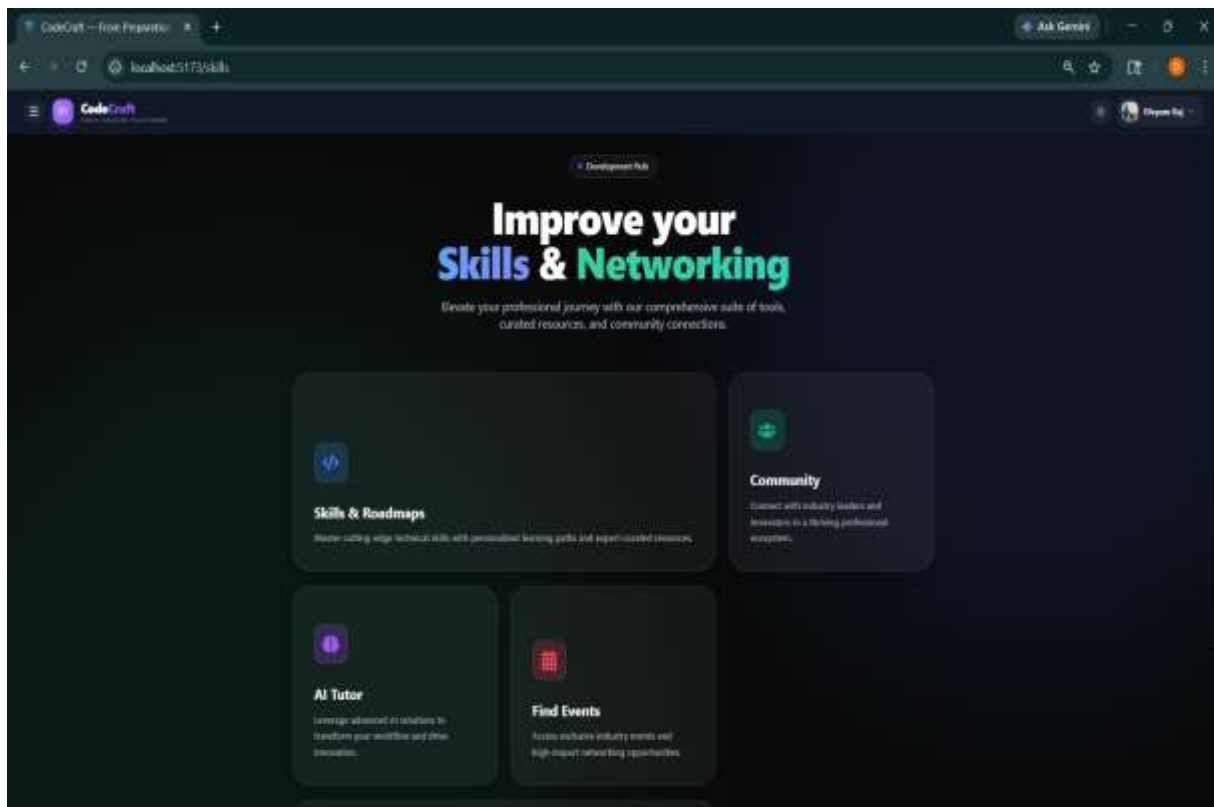
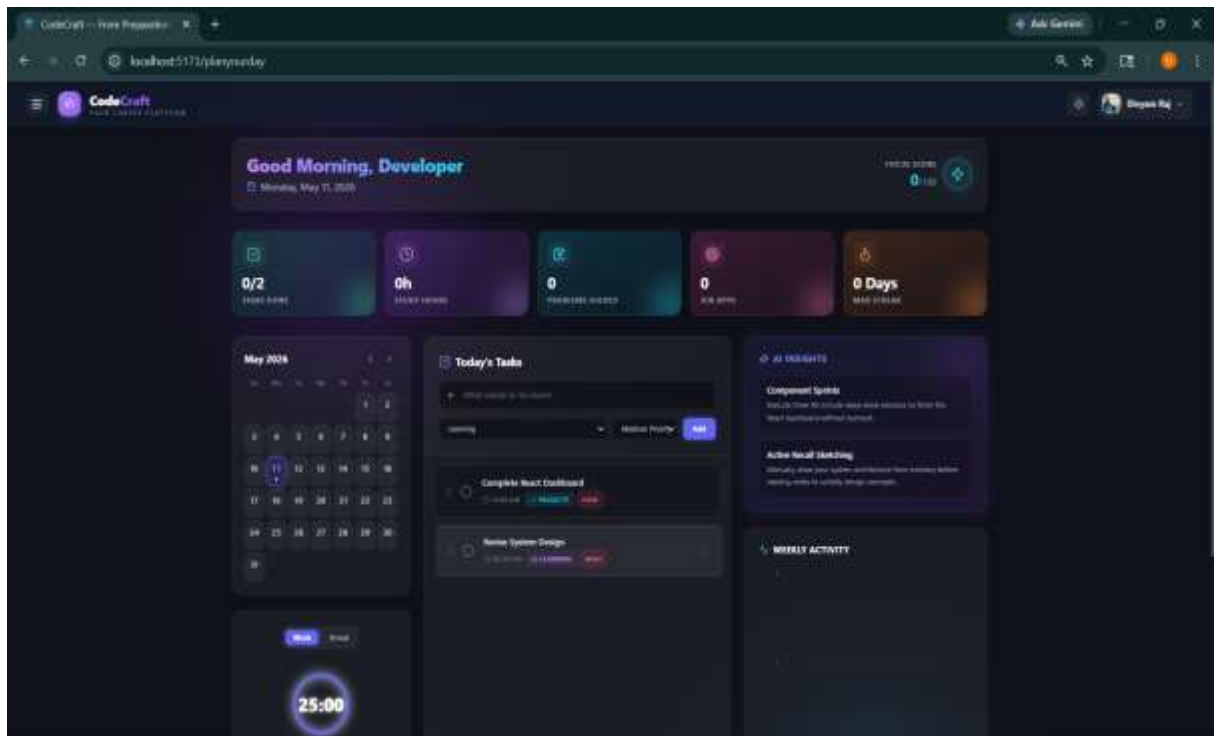


Fig. 12 – AI And Other Sections.

CHAPTER 9

TESTING

9.1 Introduction

Testing is a critical phase in the software development lifecycle that ensures the system functions as intended and meets user requirements. It involves verifying individual components, validating system behaviour, and identifying potential defects before deployment.

For CodeCraft , testing was performed at multiple levels to ensure that authentication, question generation, AI evaluation, database operations, and user interface interactions work correctly. The testing strategy focused on functionality, usability, security, and performance to deliver a reliable interview preparation platform.

9.2 Objectives of Testing

The primary objectives of testing CodeCraft are:

- To verify that all functional requirements are implemented correctly
- To ensure smooth interaction between frontend, backend, AI services, and database
- To validate secure authentication and session handling
- To identify and eliminate system errors and edge cases
- To ensure consistent AI evaluation output
- To confirm that the system performs well under normal usage

9.3 Testing Strategy

CodeCraft follows a multi-level testing strategy, ensuring comprehensive coverage of the application.

Levels of Testing Performed:

- Unit Testing
- Integration Testing
- System Testing
- Functional Testing
- Security Testing
- User Acceptance Testing

□

9.4 Unit Testing

Unit testing focuses on testing individual modules or components in isolation.

Modules Tested:

- Authentication logic
- Question generation logic
- API request handlers
- Database interaction functions
- AI evaluation response processing

9.5 Integration Testing

Integration testing verifies the interaction between different system modules.

Modules Integrated:

- Frontend with backend API routes
- Backend with AI evaluation service
- Backend with database
- Authentication layer with protected routes

Focus Areas:

- Correct data flow between modules
- Response formatting consistency □ Error propagation handling

9.6 System Testing

System testing evaluates the application as a whole in a real-world environment.

Scenarios Tested:

- User registration and login
- Interview category selection
- Question generation
- Answer submission
- AI-based evaluation
- Session history retrieval □ Logout functionality

9.7 Functional Testing

Functional testing ensures that every feature works according to specifications.

Key Functional Areas Tested:

- Secure login and signup
- Dynamic routing and rendering using Next.js
- Answer evaluation using Gemini API
- Data storage and retrieval using MongoDB

9.8 Security Testing

Security testing was conducted to ensure that user data and system resources are protected.

Security Measures Tested:

- Unauthorized access prevention
- Session validation
- Token-based authentication
- API route protection
- Secure database access

9.9 Performance Testing

Performance testing ensures that the system responds efficiently under normal usage.

Performance Metrics Evaluated:

- Page load time
- API response time
- AI evaluation response delay
- Database query execution time

9.10 User Acceptance Testing

User Acceptance Testing (UAT) was conducted by allowing sample users to interact with the system.

Feedback Observations:

- Interface was easy to understand
- Feedback was helpful and detailed
- Navigation was smooth
- AI evaluation was relevant

9.11 Test Case Table

Test Case ID	Test Description	Input	Expected Output	Result
TC01	User Login	Valid credentials	Dashboard displayed	Pass

TC02	Invalid Login	Wrong credentials	Error message	Pass
TC03	Category Selection	Valid category	Question generated	Pass
TC04	Answer Submission	Valid answer	Evaluation shown	Pass
TC05	Empty Answer	No input	Validation error	Pass
TC06	Session History	User ID	History displayed	Pass
TC07	Logout	Click logout	Session terminated	Pass

9.12 Error Handling and Validation Testing

The system was tested for various edge cases such as:

- Empty input submissions
- Network delays
- AI API failures
- Database connection issues

CHAPTER 10

RESULTS AND ANALYSIS

10.1 Introduction

This chapter presents the results obtained after implementing and testing the CodeCraft application. It analyses system behaviour, performance outcomes, user interaction results, and the effectiveness of AI-driven interview evaluation. The analysis focuses on how well the system meets the objectives defined earlier and how it improves the interview preparation process compared to traditional methods.

The results are evaluated based on functional accuracy, response quality, usability, performance, and overall user experience.

10.2 Functional Results

After successful implementation, all core functionalities of CodeCraft were executed and verified.

10.2.1 Authentication Results

- Users were able to sign up and log in securely.
- Sessions were maintained correctly across pages.
- Unauthorized users were restricted from accessing protected routes.

10.2.2 Question Generation Results

- Interview questions were generated dynamically based on selected categories.
- Repetition of questions was minimized.
- Questions matched interview-level difficulty and relevance.

10.2.3 Answer Submission and Evaluation Results

- Users were able to submit long-form answers without interface limitations.
- Answers were processed correctly by the backend.
- AI-generated feedback was returned in a structured and readable format.

10.3 Analysis of AI Feedback Quality

The AI evaluation focused on multiple qualitative aspects of user responses.

10.3.1 Accuracy Analysis

- Correct technical concepts were recognized and highlighted.
- Partial answers received constructive suggestions rather than rejection.

10.3.2 Clarity and Communication Analysis

- Feedback pointed out unclear explanations.
- Suggestions were provided for improving sentence structure and flow.

10.3.3 Depth of Explanation

- Answers lacking examples or reasoning were flagged.
- Users were encouraged to elaborate further.

10.4 Performance Analysis

10.4.1 Response Time

- Page navigation was smooth due to optimized rendering.
- API responses were delivered within acceptable time limits.
- AI evaluation responses, though computationally intensive, remained within practical delays.

10.4.2 Database Performance

- Session records were saved correctly.
- History retrieval was fast even with multiple entries.

10.5 User Interaction Analysis

The user interface was evaluated based on usability and accessibility.

Observations:

- Users found the dashboard intuitive.
- Navigation between pages was simple.
- Feedback presentation was clear and readable.
- The system reduced interview anxiety by encouraging repeated practice.

10.6 Comparison with Traditional Preparation Methods

Parameter	Traditional Methods	CodeCraft
-----------	---------------------	-----------

Question Type	Static	Dynamic
Feedback	Self-assessed or human	AI-generated
Availability	Limited	24/7
Personalization	Low	High
Performance Tracking Parameter	Rare Traditional Methods	Available CodeCraft
Cost	Often high	Low / Free

10.7 Achievement of Project Objectives

All objectives defined in earlier chapters were achieved:

- AI-based evaluation implemented successfully
- Secure authentication integrated
- Scalable architecture deployed
- Responsive and user-friendly UI developed ☐ Performance tracking enabled

10.8 Limitations Observed

Although the system performed well, certain limitations were observed:

- AI responses may vary based on prompt structure
- Evaluation depends on text input only
- Performance may reduce under very high concurrent usage

CHAPTER 11

CONCLUSION

The rapid evolution of recruitment processes has significantly increased the need for efficient, structured, and personalized interview preparation methods. Traditional approaches, which rely heavily on static resources or limited human feedback, often fail to provide continuous guidance and objective evaluation. In this context, CodeCraft successfully addresses the challenges associated with conventional interview preparation by offering an AI-powered, automated, and user-centric platform.

CodeCraft integrates modern web technologies with artificial intelligence to simulate a realistic interview environment. The system allows users to practice interview questions dynamically, submit detailed answers, and receive instant, structured feedback generated through advanced language models. Secure authentication ensures data privacy, while efficient database management enables users to track their progress over time.

The project demonstrates that artificial intelligence can play a significant role in enhancing learning outcomes by providing unbiased evaluation and personalized improvement suggestions. The modular architecture, scalable design, and clean user interface further contribute to the system's effectiveness and reliability.

Overall, CodeCraft fulfils its intended objectives by delivering a robust, accessible, and intelligent interview preparation platform. The project successfully bridges the gap between theoretical learning and real-world interview readiness, making it a valuable tool for students and job seekers.

CHAPTER 12

FUTURE SCOPE

12.1 Future Enhancements

Although CodeCraft meets its current objectives, there is significant scope for future improvement and expansion. With continuous advancements in artificial intelligence and web technologies, the platform can be enhanced to support more complex and realistic interview preparation scenarios.

12.1.1 Voice-Based Interview Simulation : Future versions of CodeCraft can include voice-based interviews where users respond verbally instead of typing answers. Speech-to-text processing combined with AI evaluation can simulate real interview conditions more accurately.

12.1.2 Video Interview Analysis : Integration of video-based interviews can allow analysis of non-verbal communication such as facial expressions, eye contact, and confidence levels. This would provide a more comprehensive assessment of candidate performance.

12.1.3 Advanced Performance Analytics : The system can be enhanced with graphical analytics such as:

- Performance trend charts
- Category-wise strength and weakness analysis
- Improvement recommendations based on historical data

12.1.4 Resume and Profile Evaluation : Future enhancements may include resume analysis and role-based interview preparation, where questions are generated based on a user's resume and target job role.

12.1.5 Multi-Language Support : Adding support for multiple languages would make the platform accessible to a wider audience, especially non-native English speakers.

12.1.6 Institution-Level Deployment : CodeCraft can be scaled for use by educational institutions and training centres, enabling batch-level monitoring and performance evaluation for students.

12.2 Scalability and Commercial Potential

With cloud deployment and AI-driven evaluation, CodeCraft has strong potential to evolve into a commercial interview preparation solution. Subscription-based models, institutional licensing, and premium analytics features can be explored in the future.

12.3 Summary

The future scope of CodeCraft is extensive and promising. With continuous refinement and integration of advanced technologies, the platform can evolve into a comprehensive interview preparation ecosystem that supports users at multiple levels of their career journey.

CHAPTER 13

REFERENCES

- Sommerville, I., *Software Engineering*, Pearson Education.
- Pressman, R. S., *Software Engineering: A Practitioner's Approach*, McGraw-Hill.
- Next.js Documentation – Official Reference Guide.
- React.js Documentation – Official React Guide.
- MongoDB Documentation – Database Design and Operations.
- Authentication System Documentation (Clerk).
- Artificial Intelligence and Large Language Model Research Papers. □ Web Development Best Practices – MDN Web Docs.
- Ian Sommerville, *Software Engineering*, Pearson Education.
- Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.
- Efraim Turban, *Artificial Intelligence and Decision Support Systems*, Pearson.
- Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson Education.
- Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley.
- Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall.
- Tailwind CSS Official Documentation.
- ShadCN UI Component Documentation.
- Node.js Official Documentation.
- Mongoose ODM Documentation.
- Google Gemini API Documentation.
- MDN Web Docs – Web Development Best Practices.
- Vercel Deployment Documentation.
- JavaScript Info – Modern JavaScript Tutorial.
- OpenAI and Google Research Papers on Large Language Models (LLMs).
- IEEE Research Papers on AI-Based Learning and Interview Preparation Systems.
- ACM Digital Library Research Articles on Intelligent Assessment Systems.